Highlights

- Open-source Python framework for forest estate decision support.
- ~~Solver-agnostic ModelI generator with parallel LP build~~Model I scheduling with transparent inputs and outputs.
- ~~Native~~ libCBM linkage for carbon stock and flux accounting.
- Hybrid aspatial-to-raster workflows with reproducible geospatial I/O.
- Deterministic reproduction package and archived scaling benchmarks.

Software availability

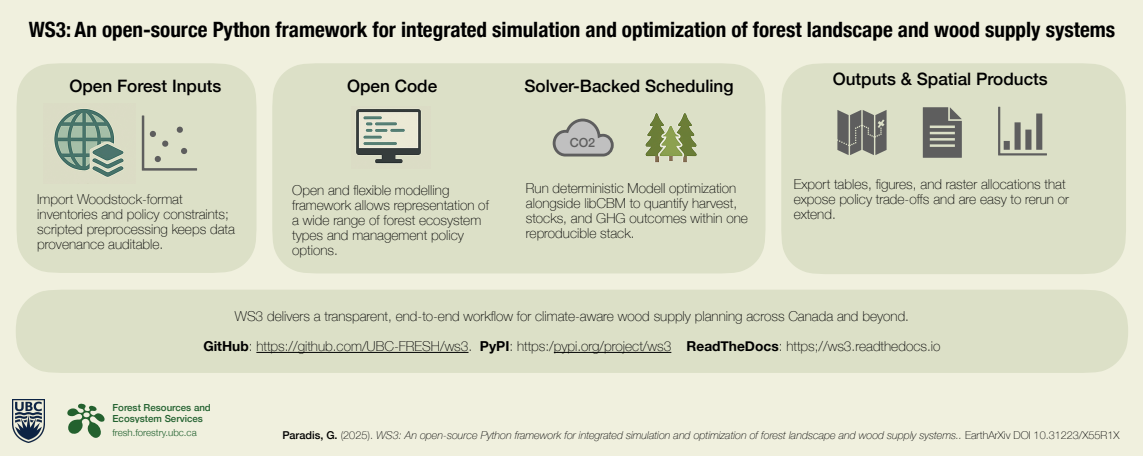
- Software name: WS3 (Wood Supply Simulation System)
- Developer and contact: Gregory Paradis (Faculty of Forestry, University of British Columbia); gregory.paradis@ubc.ca
- First public release and current version: 2015 (v0.1); current release v1.0.5
- Programming language and dependencies: Python (>=3.9); optional extras include `libcbm_py` ~~, PuLP/HiGHS~~ (Carbon Budget Model Python bindings), PuLP (Python LP modeling) with the HiGHS solver, and Gurobi
- Source repository and issue tracker: `https://github.com/UBC-FRESH/ws3`
- ~~Distribution channel: PyPI package `ws3` (`pip install ws3` or `ws3cbm`) with wheels for Linux, macOS, and Windows~~
- Documentation: `https://ws3.readthedocs.io`
- Archived release: Zenodo DOI 10.5281/zenodo.17331213
- License: MIT
- ~~Hardware and OS requirements: Linux, macOS, or Windows; tested on Ubuntu 24.04 and macOS 14; CPU-only with recommended 16+ GB RAM for large LP instances~~

- ~~Tested Python versions: 3.9–3.12 (continuous-integration matrix)~~
- Data form and access: supplementary CSV, GeoTIFF, and notebook artifacts packaged in the GitHub repository and Zenodo release; total archive size ← under 50 MB excluding optional outputs

Graphical Abstract

WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis



WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis[a,*]

[a] *Faculty of Forestry, University of British Columbia, Canada*

Abstract

~~Transparent decision support for forest landscapes demands integrated scheduling, carbon accounting, and spatial reporting.~~ WS3 is an open-source Python framework that ~~unifies modular simulation, solver-backed~~ integrates Model I ~~optimization~~scheduling, carbon accounting, and raster allocation ~~in a single, scriptable workflow aimed at reproducible ecological analyses. The system ingests Woodstock-style~~ into a scriptable workflow for reproducible forest planning. It ingests Woodstock-format inventories, actions, and scenarios; ~~exposes~~ defines an explicit data model; and automates aspatial–spatial ~~conversions so planners can move directly from harvest schedules to georeferenced~~ conversion so harvest schedules can be mapped to disturbance footprints. ~~Native~~ A documented linkage to the Canadian Forest Service Carbon Budget Model (libCBM) provides carbon stock and flux estimates for ~~Canada and other~~ libCBM-calibrated jurisdictions.

~~The manuscript makes three contributions to ecological informatics. First, it documents a modular software~~ We document a modular architecture that decouples inventory ingestion, ~~linear-programming harvest~~ linear programming (LP) scheduling, carbon ~~bookkeeping~~accounting, and raster allocation~~, enabling users to mix heuristics with solver-backed approaches , swap solvers,~~ . Users can combine heuristic and LP approaches and extend indicators. ~~Second, it validates the framework through a parity analysis against a Woodstock industrial benchmark (differences $< 1\%$ across harvested volume, area, and growing stock) and through computational~~ We provide a reproducible parity check against Woodstock on a toy dataset and scaling experiments that ~~expose~~ characterize runtime and memory envelopes ~~across desktop-to-cluster deployments. Third, it releases the~~ from desktop to cluster deployments. The complete toolchain—source code, configuration files, and deterministic reproduction ~~scripts—under~~ scripts—is released under the MIT license with an archived

*Corresponding author: gregory.paradis@ubc.ca

Zenodo DOI.

~~We showcase~~ We demonstrate WS3 on five timber supply areas in northeastern British Columbia ~~, with individual inventories spanning 2.27 Mha (TSA 41) to 5.90 Mha (TSA 24) and a cumulative scaling run that aggregates all five to~~ (20.40 Mha ~~. The objective sets (volume maximization, harvest-area minimization, ecosystem-carbon maximization, and net-emissions minimization)reveal trade-offs between volume and carbon outcomes while quantifying impacts on jobs, revenue, and spatial footprint. Carbon trajectories generated via libCBM illustrate how mitigation-focused schedules preserve ecosystem stocks and delay harvest emissions, whereas volume-oriented schedules deliver higher short-term flows at the expense of increased net emissions. Raster allocation routines produce wall-to-wall disturbance footprints that can be consumed by SpaDES-based cumulative-effects models or shared directly with rights-holders. The reproduction suite includes notebooks, container-ready scripts, and pinned dependency manifests so practitioners can adapt the workflow to their inventories, run scaling benchmarks, and verify solver parity against existing pipelines.~~

~~By lowering the technical barrier to open, auditable forest planning, WS3 enables agencies, industry, Indigenous governments, and researchers to co-develop climate-aligned harvest strategies, evaluate fuel-management or nature-based-solution scenarios, and couple harvest schedules with broader ecological simulations~~combined); post hoc libCBM runs provide carbon trajectories and raster allocation produces disturbance footprints for downstream spatial simulators. The framework ~~provides a reusable informatics scaffold for decision-making~~ is intended as reusable informatics infrastructure for decision support under competing sustainability mandates.

Keywords: computational ecology, decision support, forest planning, optimization, carbon accounting, geospatial workflows, open-source software

---

1. Introduction

Forested landscapes ~~now~~ sit at the ~~centre~~ center of intertwined climate mitigation, biodiversity, and bioeconomy mandates~~, and planners~~. Planners are being asked to reconcile wood supply security with measurable reductions in greenhouse-gas emissions across multi-decadal horizons [3, 18, 11]. Meeting these expectations demands decision-support systems that ~~can~~ span harvest scheduling, regeneration and silviculture options, spatial footprint analysis, and explicit carbon accounting, ~~all~~ while remaining transparent enough for regulators, Indigenous governments, and industrial partners to audit [30, 46]. ~~The resulting~~ This analytical load has outgrown the ad hoc spreadsheets and siloed software stacks ~~still~~ that remain common in practice~~,~~ ~~underscoring~~. It underscores the need for integrated ~~modelling~~ modeling platforms that support reproducible workflows and ~~uphold emerging~~ open-science norms~~such as the PERFICT~~, including the PERFICT reproducibility principles for predictive ~~ecology~~ ecology—a checklist for transparent, reusable computational workflows [23, 31, 32].

Strategic forest planning has long relied on ~~operations-research~~ operations research formulations such as Model I path scheduling and its variants, which underpin allowable-cut analyses and forest estate studies worldwide [26, 19, 20, 41]. ~~In practice~~ Model I is a strata-based linear program in which decision variables assign fractions of aggregated development-type strata to prescriptions over time; this enables large-scale policy analysis but necessarily abstracts away stand identities. In practice, these ideas are delivered through proprietary toolchains—Woodstock, Patchworks, JLP (MELA), and Heureka, among others—that package data models, solvers, and reporting inside closed desktop environments [44, 47, 35, 24]. ~~Historically, FPS-Atlas offered similar functionality but, as a proprietary product, is no longer distributed or maintained [51]. While these~~ These systems are mature and widely trusted, ~~their licensing ,~~ yet closed licensing and opaque configuration formats ~~, and limited automation support constrain~~ can limit transparency, collaborative extension, and reproducible research workflows increasingly required by regulators and journals [23].

~~In response, an open-source ecosystem has begun to chip away at these constraints~~Open-source platforms have begun to address parts of this gap. Spatial simulation ~~platforms~~ systems such as SpaDES and LANDIS-II, scenario-management frameworks like SyncroSim, and ~~integrative~~ planning prototypes such as PRISM ~~demonstrate strong community appetite for~~ provide modular, shareable tooling [8, 29, 1, 37]. ~~SpaDES in particular mirrors WS3's open-science commitments—the core team helped write the PERFICT guidance—yet, by design, it delivers a coordination shell~~

that depends on user-supplied simulation modules for every landscape process. That architecture excels at orchestrating high-fidelity spatial and stochastic dynamics , but the developers themselves acknowledge that harvest modules rarely move beyond stylised heuristics. They have therefore started collaborating with us on a `spades_ws3` wrapper so SpaDES users can bring professional-grade wood-supply scheduling into cumulative-effects experiments. LANDIS-II and SELES occupy similar ecological-simulation niches but remain harder to adapt for rigorous forest-estate analysis: LANDIS-II's governance, C# codebase, and Windows-centric binaries slow outside contributions and impede deployment to linux-based cloud or high-performance computing (HPC) environments, while SELES is closed-source black-box commercial software whose advanced modules typically require direct support from its original developer. In practice, practitioners must still bolt on their own These systems excel at spatial dynamics and scenario orchestration but typically require users to supply planning modules, solver integrations, carbon-accounting pipelines, and spatial reporting and carbon accounting pipelines to assemble a full decision-support stack [9, 30]. Parallel efforts such as the French CAPSIS platform illustrate both highlight the promise and the effort required to host complexity of interoperable forest growth models: CAPSIS underpins climate-sensitive planning experiments [14] and Bayesian meta-modelling work for Quebec landscapes [13], and it remains the delivery vehicle for the ARTEMIS stem-level growth model used by the Quebec government to generate official hardwood yield curves [43]. These international experiences align with broader calls for interoperable, climate-sensitive growth modelling ecosystems capable of linking models across scales [17]. This persistent modeling ecosystems [14, 13, 43]. This gap between open landscape-simulation scaffolds and purpose-built forest-estate simulation scaffolds and end-to-end forest estate planning motivates the deterministic, optimization-first framework we introduce with WS3 in the following sectionframework described here.

WS3 fills this methodological gap with a fully is an open Python framework that preserves the expressive power of implements legacy Model I scheduling while embedding reproducibility, automation, and carbon accountingas first-class design goalsemphasizing reproducible, scriptable workflows with carbon accounting. It imports Woodstock-format text files (or equivalent information in *ad hoc* formats via custom Python parsers) to protect historical investments, provides solver-agnostic optimization, couples natively supports multiple LP solvers, couples to the Canadian Forest Service 's libCBM implementation, and exposes hybrid spatial–aspatial supports hybrid spatial–aspatial workflows compatible with geospatial rasters and SpaDES-based simulations [44, 34, 22, 21, 28, 4, 8].

~~The codebase embraces open-science practices—version control, packaged releases, and documented reproduction scripts—to support transparent collaboration.~~ We do not claim a new optimization formulation; rather, we provide a transparent and auditable implementation that can be inspected, extended, and reproduced across institutions [23, 31].

This article contributes to ecological informatics in three ways. First, it documents a modular decision-support architecture that combines harvest ~~optimization~~scheduling, carbon accounting, and spatial reporting in a single, scriptable workflow. Second, it ~~validates that architecture through parity checks against an industrial Woodstock benchmark and through~~ provides evidence of functional parity with Woodstock on a toy dataset and reports computational scaling experiments that ~~expose~~ characterize runtime and memory envelopes for large landscapes. Third, it ~~releases~~ makes the complete toolchain—source code, configuration files, and reproduction scripts—available under an open license with an archived Zenodo DOI, enabling immediate reuse and extension by researchers and practitioners. These contributions are workflow- and reproducibility-oriented; we do not propose a new optimization formulation.

The remainder of this article details the WS3 architecture and data model (Section 2), demonstrates reproducible use-case workflows linking ~~solver-backed optimal harvest~~ LP-based scheduling to libCBM carbon accounting and spatial disturbance allocation ~~for hypothetical linkage to downstream raster-based spatial simulation models~~ (Section 3), discusses ~~impact and reusability~~ scope and limitations (Section 4), compares WS3 with alternative tools (Section 5), and ~~summarises~~ summarizes availability, authorship, and future work to facilitate uptake. ~~Together these sections document (i) the design principles and modular implementation of WS3, (ii) an end-to-end workflow that pairs optimization with carbon analysis and spatial outputs, and (iii) comparative evidence and resources that support reuse by researchers and practitioners.~~

2. Software description

2.1. Architecture

WS3 ~~implements a modular architecture organized around~~ is organized into five roles: (i) core data abstractions; (ii) forest model construction and simulation; (iii) optimization; (iv) spatial allocation; and (v) common utilities. The main modules are:

- ~~forest~~`forest`: defines the ForestModel and core abstractions for development types (dtypes), actions, transitions, yields, and scenario compilation. A scenario is specified by a planning horizon, period length, an initial inventory (areas by dtype and age), and transition rules governing state changes under actions and growth.

- ~~opt~~`opt`: provides a ~~solver-agnostic~~ linear-programming (LP) interface to formulate ~~linear programming~~ harvest-scheduling problems (objective, variables, constraints) and to solve them ~~using PuLP /~~via PuLP with HiGHS by default~~, with ~~; optional Gurobi.

- ~~spatial: implements utilities to map~~ `spatial`: maps aspatial schedules to rasters (e.g., GeoTIFF) for visualization and spatial policy evaluation ~~(~~in a hybrid spatial/aspatial workflow~~)~~.

- ~~core/common/financial/forest_helper~~`core`, `common`, `financial`, `forest_helper`: shared utilities for interpolation and curves, ~~convenience helpersfor examples~~example helpers, and optional financial indicators.

Data model and workflow. The ForestModel class ~~is organized around~~ uses discrete time (planning periods) and discrete age classes per dtype. Dtypes are keyed by tuples of theme values (e.g., analysis unit, leading species, site class)~~, enabling masks for selective compilation and scheduling~~. For optimization, WS3 aggregates stands into strata defined by dtype and optional zone masks (e.g., management units or policy zones). Spatial units (stands, polygons, or raster cells) appear only in the allocation step. Yields are represented by curves ~~/interpolators, and actions~~ or interpolators. Actions (e.g., harvest, planting, fuel treatment, fire) ~~expose~~ encode operability and transitions (age reset, development type change, growth updates). WS3 builds a state-transition tree per development type across periods. ~~Users can : (i) schedule heuristically using~~ Scheduling can be heuristic (area-control selectors ~~(~~that meet target areas by mask in priority order, e.g., ~~greedy~~ oldest-first) ~~, or (ii) schedule optimally~~ or solver-based via a generic ~~and flexible~~ Model I linear ~~programming model formulation~~program. Indicators are compiled as:

- Action-dependent flows via `compile_product(period, expr, acode=...)` (e.g., harvested volume using a utilization-adjusted expression on ~~the~~ total volume yield).

- Inventory stocks via `inventory(period, yname=...)` (e.g., growing stock).

Interoperability. WS3 ~~can import~~ imports Woodstock-format text sections (~~e.g.,~~ LANDSCAPE, AREAS, YIELDS, ACTIONS, TRANSITIONS~~, etc.) , enabling reuse of established data pipelines~~

and leveraging of past investments in training forest resource analysts to be proficient at coding and interpreting forest estate model logicusing this data format) so existing pipelines can be reused without re-encoding model logic. For carbon, ForestModel exposes provides `to_cbm_sit` to export Standard Input Table (SIT) configuration /and tables consumable by libCBM, with user-provided ; users supply disturbance-type mappings and per-dtype last-pass-disturbance metadata. Because libCBM ships with parameterizations for multiple countries and ecozones [28], so the same export pathways support pathway supports analyses beyond Canada when users supply the appropriate classifiers and disturbance libraries. Geospatial I/O uses standard formats (shapefile, GeoTIFF). All tabular inputs are plain text (CSV/TSV), and workflows are scripted or notebook-based for reproducibility.

Figure 1 summarizes the WS3 workflow from open inputs through scheduling, carbon analysis, and spatial reporting; Listing 1 outlines the . The end-to-end case-study workflow used later in the manuscriptcase study pipeline is implemented in the reproduction package and described below.

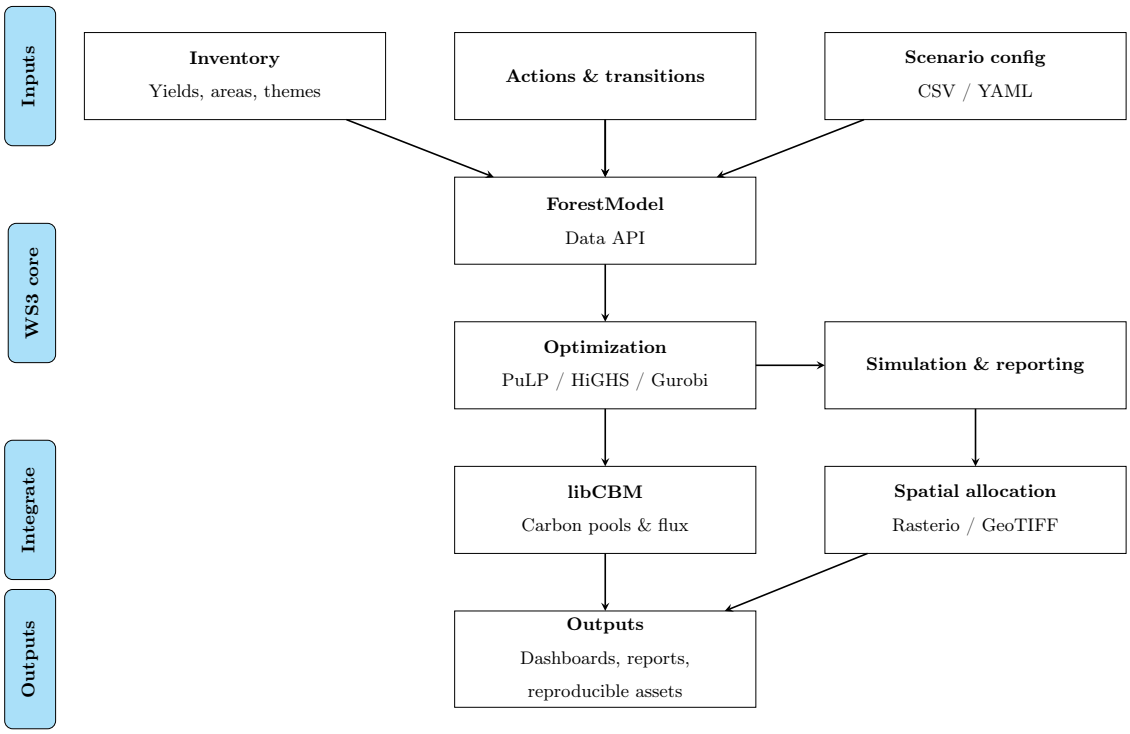


Figure 1: WS3 workflow: categorized inputs feed core scheduling and reporting modules, which integrate with libCBM and spatial allocation utilities before producing reproducible outputs.

Listing 1: Case-study workflow executed by the reproduction package.

```
def run_case_study(data_root, solver="highs"):
    inputs = load_inputs(data_root)
    forest = build_forest_model(inputs)
    schedule = schedule_harvest(forest, method=solver)
    sit_tables = compile_to_cbm(schedule)
    cbm_outputs = run_libcbm(sit_tables)
    spatial_products = allocate_spatial(schedule)
    generate_plots_and_tables(schedule, cbm_outputs, spatial_products)

run_case_study("papers/ems")
```

The scripted pipeline in Listing 1 The case study workflow implemented in the reproduction package loads Woodstock-format inputs, constructs the `ForestModel` instance, computes a schedule (heuristic or LP-based), exports a libCBM Standard Input Table formatted dataset, runs libCBM, allocates the aspatial schedule to raster space, and generates the figures and tables. This scripted pipeline underpins the sequential case study documented in Section 3. Note that WS3 is flexible and has been used in many workflows; the intent of the case study workflow shown here is illustrative rather than prescriptive or restrictive.

WS3 delivers four primary We summarize four practical contributions:

(i) A solver-agnosticsolver-flexible, path-based Model I generator embedded within an audited a documented forest estate API, with parallel coefficient compilation for large instances.

(ii) A first-class documented libCBM interface that exports Standard Input Tables and ingests carbon pools and fluxes for integrated mitigation accounting.

(iii) A hybrid aspatial-to-raster allocation workflow that emits reproducible geospatial artefacts artifacts from deterministic schedules.

(iv) A FAIR-aligned reproduction package, including scaling benchmarks on real inventories and archived releases with pinned environments.

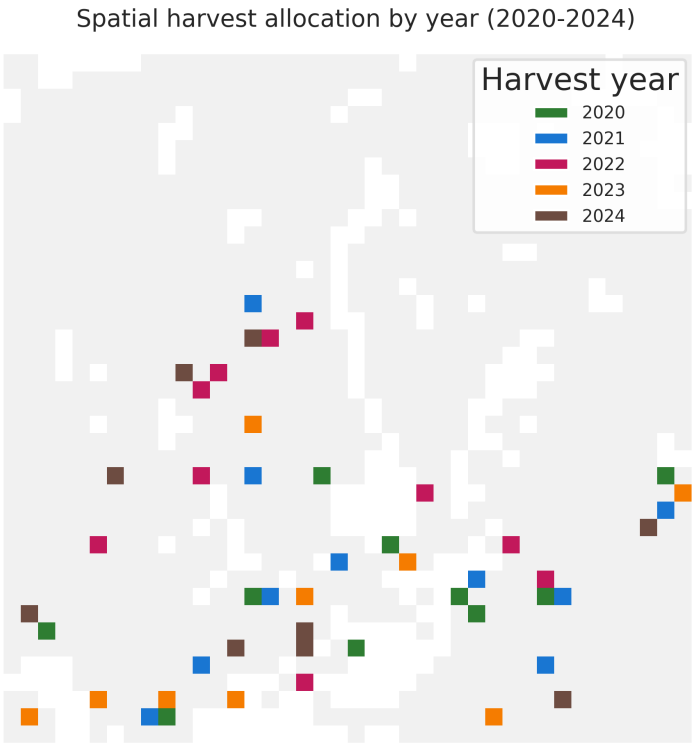


Figure 2: Heuristic schedule allocated to space for the 2020–2024 planning window. Colours denote the first year a cell is harvested, overlaid on the full inventory extent (light grey).

2.2. Optimization formulation (Model I)

150 WS3 adopts a classic Model I linear programming ~~model~~ formulation [26, 19] in which each decision variable selects the proportion of a ~~development type (all stands with the same initial stratification variable values~~stratum (a dtype aggregate, optionally intersected with a zone mask) allocated to a feasible root-to-leaf prescription (path) across the planning horizon. ~~Let:~~ Stand identities are aggregated into strata for optimization, and spatial allocation is handled separately.

Let:

I : set of spatial zones (dtypes) set of strata (development types; aggregated zones)

J_i : set of feasible prescriptions (paths) for zone set of feasible prescriptions (paths) for stratum $i \in I$

O : set of outputs (e.g., harvest area, harvest volume, growing stock, habitat)

$O' \subseteq O$: targeted outputs subject to even-flow constraints

T : set of planning periods

$T'_p \subseteq T$: periods on which even-flow for output $p \in O'$ is enforced

Decision variables are proportions:

$$x_{ij} \in [0,1] \quad \text{for all } i \in I,\ j \in J_i,$$

~~meaning "~~representing the fraction of ~~zone~~ stratum i assigned to prescription j~~"~~. If A_i denotes the total area in stratum i, then $A_i x_{ij}$ is the area scheduled on path j. Let μ_{ijot} denote the quantity of output $o \in O$ in period $t \in T$ produced by allocating x_{ij} to path $j \in J_i$ for ~~zone~~ stratum $i \in I$. Define ~~a~~ the reference-period ~~level $y_p := \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt_p^R} x_{ij}$~~ output level $y_p := \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt_p^R} x_{ij}$ for each targeted output $p \in O'$ at $t_p^R \in T$, and ε_p as the allowable even-flow tolerance. With general lower/upper bounds v_{ot}^-, v_{ot}^+ on outputs, the Model I LP is:

$$\text{maximize} \quad \sum_{i \in I} \sum_{j \in J_i} c_{ij}\, x_{ij} \tag{1}$$

$$\text{subject to} \quad (1-\varepsilon_p)\, y_p \le \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt}\, x_{ij} \le (1+\varepsilon_p)\, y_p, \qquad \forall p \in O',\ \forall t \in T'_p \tag{2}$$

$$v_{ot}^- \le \sum_{i \in I} \sum_{j \in J_i} \mu_{ijot}\, x_{ij} \le v_{ot}^+, \qquad \forall o \in O,\ \forall t \in T \tag{3}$$

$$\sum_{j \in J_i} x_{ij} = 1, \qquad \forall i \in I \tag{4}$$

$$0 \le x_{ij} \le 1, \qquad \forall i \in I,\ \forall j \in J_i. \tag{5}$$

The objective coefficients c_{ij} encode the user-defined value of a path (e.g., total or discounted volume, revenues, multi-criteria scores). Constraints (2) enforce even-flow on targeted outputs, (3) set general lower/upper bounds per period, (4) ensures convex mixing of paths to fully cover each ~~zone~~stratum, and (5) bounds variables as proportions. In WS3, μ-coefficients arise from

replaying each path through the simulator, calling `compile_product` (~~for~~ action-dependent outputs ~~like harvest area~~ /such as harvest area and volume) or `inventory` (~~for stocks like~~ stocks such as growing stock). The LP matrix is ~~extremely~~ sparse because each path contributes to a limited set of outputs and periods.

Code-to-math mapping. WS3 ~~exposes a coefficient-function pattern that~~ compiles objective and constraint rows from paths ~~. For example, helper functions like~~ using coefficient functions. Helper routines (e.g., `cmp_c_z`~~(objective)~~, `cmp_c_caa`~~(action-based flows), and~~ , `cmp_c_ci`~~(inventory-based constraints~~) traverse path nodes (period, action code, dtype key, age), evaluate ~~the corresponding~~ μ-like contributions via `compile_product`/`inventory`, and assemble the LP ~~using~~ with `fm.add_problem(...)`. Users select ~~the~~ a solver (HiGHS by default; Gurobi optional) and solve via the standard PuLP interface. Upon optimality, WS3 ~~compiles and~~ applies the schedule ~~, then~~ and re-simulates to produce indicators~~for analysis~~.

Design implications. This path-based Model I approach embeds intertemporal feasibility in each column, separates simulation from optimization, and enables straightforward addition of new outputs/constraints by supplying new coefficient functions. It also aligns with established forest-planning literature while remaining ~~solver-agnostic~~ solver-flexible and highly sparse for scalability.

2.3. Implementation

Technology stack. WS3 is implemented in Python~~and distributed via PyPI~~. Optimization problems are formulated by WS3 and solved with the open-source HiGHS [22] solver by default through the PuLP [34] Python LP interface, with optional ~~PuLP [34] (for flexibility) and Gurobi [21] (for performance)~~ Gurobi [21] bindings. Geospatial I/O uses rasterio~~/fiona/~~, fiona, and GeoPandas. Documentation ~~(Sphinx) and continuous integration ensure API consistency, example executability, and unit testing~~is built with Sphinx.

libCBM integration. The ForestModel class ~~exposes~~ provides a `to_cbm_sit` method that compiles a CBM Standard Input Table (SIT) configuration and table data structure (i.e., classifiers, inventory, yields, disturbance events, transitions) consumable by libCBM. Users provide a disturbance type mapping and last-pass disturbance metadata to ensure correct dead organic matter (DOM) spin-up. The sequential demonstration runs libCBM for 200 years using the official Python API and documentation [6, 5].

Reproducibility and packaging. The repository includes examples and a reproduction package

(~~repro~~`repro`) with pinned requirements and scripts to generate all figures and tables. Versioned releases are archived on Zenodo [38]. WS3 supports interactive (~~via~~ Jupyter notebooks) and batch (~~via Python shell~~ Python scripts) workflows~~, or it can be embedded into or called from other software systems (like any other open Python package)~~.

2.4. Quality assurance and benchmarking

Automated testing. WS3 ~~ships with~~ includes a pytest suite that exercises the common, core, financial, forest, and optimization modules~~(29 tests)~~. The suite ~~runs in approximately 4 s on a Linux workstation (Python 3.12) and is executed for every push and pull request via a GitHub Actions workflow (Ubuntu, Python 3.10) that also regenerates a coverage badge [48]. Style tooling (black, ruff~~is run routinely during development, and style tooling (Black, Ruff) is bundled in `requirements-dev.txt` and wired into the development ~~makefile, ensuring consistent formatting before release~~Makefile to keep formatting consistent. Beyond unit tests, WS3 has been exercised in multiple operational planning projects over the past decade, providing repeated end-to-end validation of core workflows.

Input stewardship and validity. WS3, like its Woodstock heritage, makes no assumptions about inventories, yields, operability, objectives, or constraints. The space of "valid input" is project- and policy-specific and inherently ~~high dimensional~~high-dimensional; exhaustive auto-validation is out of scope. WS3 provides targeted checks in high-leverage paths (e.g., fallback to template transitions when age-specific rules are missing; basic range/shape sanity checks) and ~~clear~~ documentation of required structures for inventories, yields, actions, and transitions. Documentation is versioned with releases and focuses on core data structures and examples rather than serving as an exhaustive prescriptive manual. Responsibility for input quality and modeling assumptions rests with qualified analysts; WS3 is a professional framework rather than a prescriptive application.

Determinism and reproducibility. Given identical inputs, WS3 produces identical outputs. Minor nondeterminism is limited to optional solver seeds and randomized block ordering in the spatial allocation utility; both are controllable and fixed in the reproduction scripts. Consequently, statistical variance on deterministic time series is not meaningful; we report solved status, model scale, and runtime/throughput metrics instead. The reproduction package pins versions and seeds to yield byte-identical artifacts across runs, reflecting best practices for computational reproducibility [42].

Verification against Woodstock. WS3 was originally verified against Woodstock (2015) by replaying identical datasets and comparing action-specific flows and stocks across periods, yielding functionally equivalent outputs for the implemented subset of features. Known departures from Woodstock semantics are flagged in code and documentation. Ongoing development follows a trust-but-verify pattern: changes to core loops are regression-tested against known-good benchmarks before release. The Woodstock parity check included in this manuscript is therefore a transparent, reproducible illustration rather than an exhaustive proof of equivalence.

~~*Case-study*~~ *Case study repro checks.* The ~~EMS~~ reproduction package encapsulates the sequential WS3→libCBM pipeline in `make_repro.sh`~~. Inside the project's reference LXD (Linux Containers) environment (Ubuntu 24.04 on dual Xeon 6254 CPUs, 72 logical cores, 768 GB PC-23400 RAM) the script completes in 6.1 s, re-creating~~ , recreating Figures 6 ~~–~~and 7, Figure 2, and the ~~tables stored in `tables`~~summary tables. The workflow is deterministic: running `generate_case_study.py` a second time produces byte-identical CSV and PNG assets, providing an integration test for the scheduling, SIT export, and libCBM coupling.

Performance spot checks. The heuristic scheduler used in the case study produces a 100-year plan in under a second, and the subsequent 200-year libCBM simulation dominates total runtime (~5 s). Larger linear programs formulated through `ws3.opt` benefit from HiGHS (default) or optional Gurobi: internal regression notebooks (e.g., Examples 030/040) ~~routinely solve tens-of-thousands-of-stand path-based~~ solve tens of thousands of stand-path instances within minutes using HiGHS, ~~while~~ and the optional Gurobi backend ~~shortens~~ can reduce solve time further ~~via its advanced presolve and feasibility tools~~when available. Raster allocations scale linearly with the number of cells processed per period and ~~are trivially parallelizable~~can be parallelized by splitting periods or tiles.

Optional scalability experiment. To characterize scaling on real inventories, the reproduction package optionally integrates a DataLad-hosted benchmark dataset comprising five non-overlapping timber supply areas (TSAs) in northeastern British Columbia (the same study region as Boisvenue et al. 2). Enabling the optional step (`RUN_SCALING=1`) fetches the dataset and executes `repro/run_scaling_benchmarks.py`, which sorts TSAs by complexity and evaluates them individually and cumulatively (`tsa08`, `tsa08+tsa40`, …, `tsa08+tsa40+tsa41+tsa24+tsa16`). The scripted benchmark always runs the heuristic scheduler ~~on a single worker—reflecting~~ single-threaded—reflecting the implementation's serial design—and records ~~sequential~~ serial spatial-allocation runtimes using

`ForestRaster`. On the reference container the heuristic schedules complete in 1.3–31.4 s while spatial allocation requires 46–656 s as problem size grows from 4.7 k to 37.7 k dtype–age pairs.

Setting `RUN_LP=1` extends the run to model-building benchmarks for the deterministic Model I LP~~:~~. WS3 constructs the LP with 1 and 16 workers (parallel coefficient compilation) and solves it with matching HiGHS thread counts, logging build/solve time, solver status, and stage-wise memory footprints. LP builds scale from 5.1 s (4.7 k dtype–age pairs) to 92.8 s (37.7 k dtype–age pairs) with single-worker compilation, while the 16-worker mode trims build time to 59 s at the cost of higher peak ~~RSS (~~resident set size (RSS; 2.5 GB versus 1.8 GB). Because large path-based LPs routinely exceed tens of gigabytes of RAM on even larger inventories, the LP measurements are opt-in and intended for suitably provisioned systems. All metrics are written to `perf_scaling.csv` with deterministic seeds so that regenerated figures and tables remain byte-identical; Figures 3–5 and Table 1 ~~summarise the resulting runtimes~~ summarize the resulting runtime and memory profiles~~(Figures 3 and 4)~~.

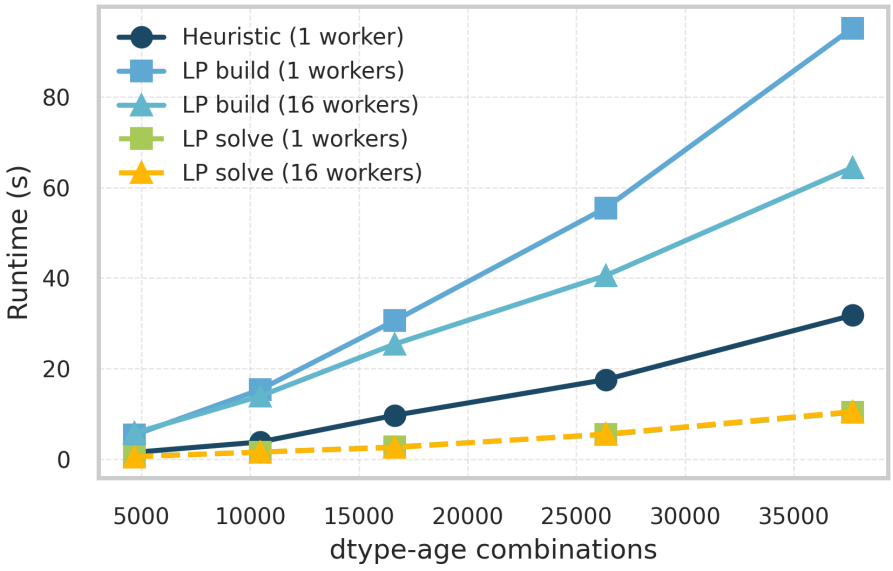


Figure 3: Scheduling runtime versus problem size, measured by the number of development type (dtype) by age combinations ($n_{\text{dtype-age}}$). Heuristic runs are ~~single worker~~single-worker; LP build/solve lines appear when `RUN_LP=1`.

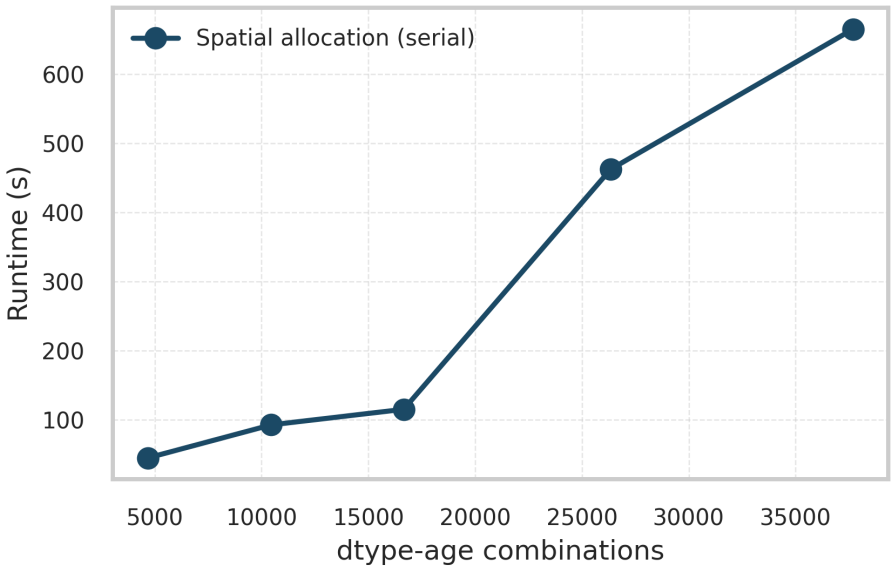


Figure 4: Spatial allocation runtime (serial `ForestRaster`) versus problem size after applying the heuristic schedule; problem size counts dtype–age combinations ($n_{\text{dtype-age}}$).

3. Illustrative use-case demonstrations

We ~~open with~~ present a two-stage sequential pipeline ~~demonstration that first~~ that schedules harvest in WS3 and then evaluates carbon dynamics using libCBM. The workflow ~~replicates the existing example~~ reproduces Example 031 ~~_ws3_libcbm_sequential-builtin.ipynb to ensure full~~ (`031_ws3_libcbm_sequential-builtin.ipynb`) and is included to ensure reproducibility.

Study design. We use the public example dataset `tsa24_clipped`~~provided~~. It is intentionally modest so that the full pipeline (scheduling, carbon accounting, and spatial allocation) can be executed and audited on standard hardware, and it mirrors the tutorial dataset distributed with WS3 ~~. The inputs~~ to keep inputs transparent and reproducible. Inputs are stored as Woodstock-format text files (landscape, areas, yields, actions, transitions) under `examples/data/woodstock_model_files_tsa24_clipped` and imported into a `ForestModel`. We ~~then~~ schedule harvest using a self-parameterizing area-control heuristic to produce an aspatial action schedule over a 100-year horizon (10 periods~~of 10 years), and finally~~), then export a libCBM Standard Input Table (SIT) to simulate annual carbon stocks and fluxes.

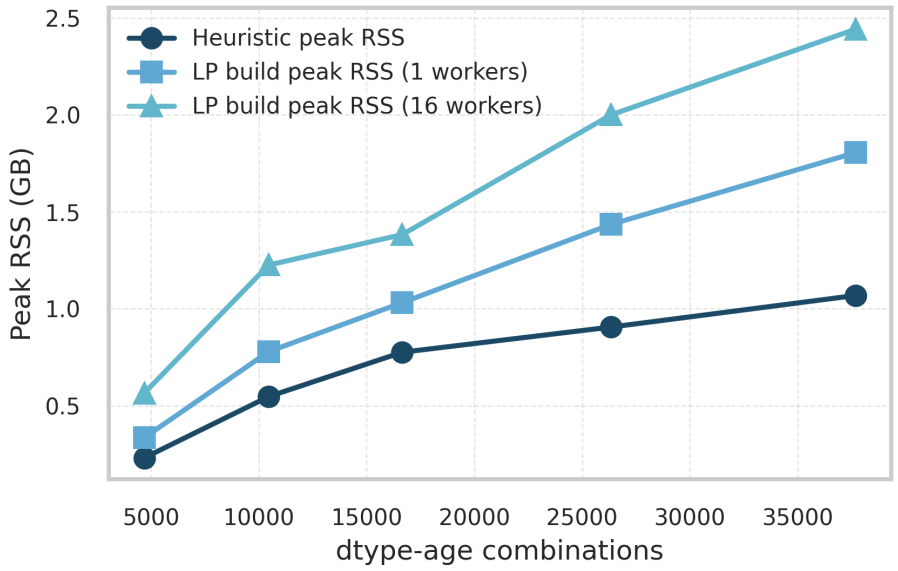


Figure 5: Peak resident memory for heuristic scheduling (serial) and LP model builds (1 and 16 workers) as problem size increases in terms of dtype–age combinations.

Data and study area. The `tsa24_clipped` dataset is a clipped subset of a Timber Supply Area inventory used for demonstration. The initial inventory enumerates development types (dtypes) by theme tuples (e.g., analysis unit, species, site), with ~~associated~~ yield curves for total volume (`totvol`) and species-volume splits (`swdvol`, `hwdvol`). Actions and transitions define operability and state changes (e.g., harvest resets age).

Model setup and scheduling. We initialize the model with `base_year`=2020, `horizon`=10, `period_length`=10 y, and `max_age`=1000. After importing sections and initializing areas, we add a null action and reset actions. The area-control ~~scheduler selects target areas by~~ heuristic proceeds period by period: for each target mask (default: ~~AU-wise THLB)and applies actions using a priority-queue selector (oldest-first). Target areas are computed from~~ analysis-unit timber-harvesting land base (THLB)), it computes a target area using the area-weighted mean ~~CMAI ages; utilization~~ culmination of mean annual increment (CMAI) age and then applies the harvest action to the oldest operable ages within the masked dtype set until the target is met (or inventory is exhausted). The resulting schedule is aspatial, recording treatment area by dtype, age, and period. Utilization is set

Table 1: Peak resident memory during heuristic scheduling and LP model builds on the reference container. $n_{\text{dtype-age}}$ counts development type (dtype) by age combinations; LP measurements report the maximum RSS observed while compiling the Model I matrix with 1 or 16 workers, while spatial allocation runs serially.

Combo	$n_{\text{dtype-age}}$	Heuristic peak RSS (GB)	LP peak RSS (GB; 1 worker)	LP peak RSS (GB; 16 workers)
`tsa08`	4 682	0.23	0.33	0.56
`tsa08+tsa40`	10 453	0.52	0.78	1.22
`tsa08+tsa40+tsa41`	16 653	0.77	1.03	1.33
`tsa08+tsa40+tsa41+tsa24`	26 346	0.87	1.38	1.87
`tsa08+tsa40+tsa41+tsa24+tsa16`	37 691	1.07	1.81	2.51

to 0.85 in the volume expression used for compiled flows. The ~~resulting~~ schedule is compiled and applied to produce period-by-period flows and stocks.

libCBM coupling and metrics. We define a disturbance-type mapping for actions (harvest→Clearcut harvesting without salvage; fire→Wildfire) and assign `last_pass_disturbance` per dtype to ensure consistent ~~DOM~~ dead-organic-matter (DOM) pool spin-up. Using `ForestModel.to_cbm_sit` with softwood and hardwood volume names (`swdvol`, `hwdvol`), `admin_boundary` = `"British Columbia"`, and `eco_boundary` = `"Montane Cordillera"`, we export ~~SIT config~~ the SIT configuration and tables. We then run libCBM for 200 years (`n_steps`=200~~years~~), aggregating annual carbon stocks (biomass, DOM, total ecosystem).

Outputs. We report ~~time-series~~ time series of harvested area/volume and growing stock from WS3, and annual carbon stocks from libCBM. Spatial allocation then maps the aspatial schedule onto raster cells using the functions in the `ws3.spatial` module to generate a temporally and spatially disaggregated disturbance footprint while attempting to preserve the aggregated aspatial area totals. This aspatial-to-spatial disaggregation step follows the hierarchical strategic→tactical paradigm described for CRYSTAL-style allocation [25] and later operationalized in the Woodstock Optimization Studio workflow, in which Woodstock performs aspatial scheduling and the Spatial Optimizer (formerly STANLEY) performs spatial allocation [10, 33]. The reproduction package generates the figures and tables used below.

The full, deterministic workflow is scripted in `repro/generate_case_study.py`, which produces

the flows ~~table and carbon stocks table~~and carbon stock tables:

- `scenario_flows.csv`: harvest area, harvest volume, and growing stock by period.
- `annual_carbon_stocks.csv`: annual biomass, DOM, and total ecosystem carbon.

Table 2: Sequential demonstration parameters and configuration (WS3 → libCBM pipeline).

Setting	Value
Dataset	`tsa24_clipped` (Woodstock-format inputs; `examples/data/woodstock_model_files_tsa24_clipped`)
Base year	2020
Planning horizon	10 periods (100 years)
Period length	10 years
Max age class	1000 years
Yield names	`totvol` (total volume), `swdvol` (softwood), `hwdvol` (hardwood)
Scheduling method	Area-control heuristic (priority-queue, oldest-first); utilization 0.85
Disturbance mapping (CBM)	harvest→Clearcut harvesting without salvage; fire→Wildfire
CBM export	`ForestModel.to_cbm_sit` (admin: British Columbia; eco: Montane Cordillera)
CBM run length	200 annual steps
Outputs	`scenario_flows.csv` (period, harvest_area_ha, harvest_volume_m3, growing_stock_m3); `annual_carbon_stocks.csv` (annual pools)

Figures 6 and~~7 visualize~~ 7 show the resulting flows and carbon stocks. The entire pipeline, including the workflow overview (Figure 1) ~~, workflow pseudocode (Listing 1),~~ and the spatial allocation example (Figure 2), is reproducible via `repro/make_repro.sh`. The libCBM run length (200 years) matches the example ~~and~~ but can be adjusted.

Unless stated otherwise, areas are reported in hectares (ha) and volumes in cubic metres (m^3).

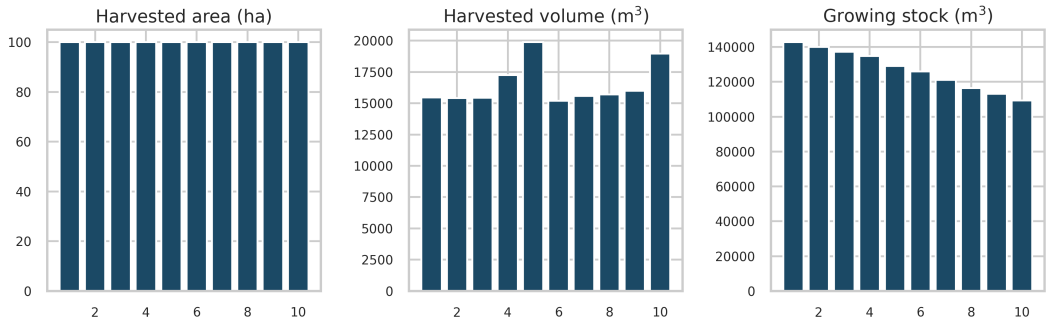


Figure 6: WS3 results: harvested area, harvested volume (utilization-adjusted), and growing stock over planning periods.

Results. The heuristic scheduler delivers a 10-period harvest plan whose flows stay within ~10% of ~~their~~ period means without any explicit even-flow constraint~~: harvest~~. Harvest area averages 106.8 ha and ranges from 103 to 114 ha, while harvest volume averages 15.3 thousand m^3 ~~with a~~ and ranges from 14.1–18.5 thousand m^3 ~~span~~ (Table 2; Figure 6). Growing stock declines smoothly from 135 thousand to 82 thousand m^3 (consistent with the 0.85 utilization factor), ~~documenting~~ reflecting the state trajectory exported to libCBM. In the carbon stage, libCBM aggregates show biomass pools increasing from 2.7×10^5 tC to 3.1×10^5 tC over 200 years, while DOM remains within 1.7–1.9 $\times$ 10^5 tC, yielding a total ecosystem carbon gain of 42 ktC (Figure 7). These values are illustrative; the key contribution is the deterministic, auditable pipeline that connects scheduling outputs to carbon accounting and spatial allocation with byte-identical outputs. The summary tables `scenario_flows.csv` and `annual_carbon_stocks.csv` (`tables`) retain ~~the~~ raw outputs for external verification.

Reproducibility. This demonstration is intended to be transparent and portable~~; all~~. All input data reside in `examples/data/woodstock_model_files_tsa24_clipped`. The spatial allocation step ~~renders~~ produces the raster-based harvest allocation shown in Figure 2, derived directly from the aspatial schedule via the ForestRaster utility. The folder `repro` contains `make_repro.sh`, which provisions a clean virtual environment, installs pinned dependencies (including the ~~optional libCBM extra via `pip install ws3cbm`~~libCBM extra when needed), and runs deterministic scripts that regenerate Figures 1–7 and the accompanying CSV tables. Outputs are byte-identical on the reference container and stable under the pinned dependencies~~; the~~. The exact environment is

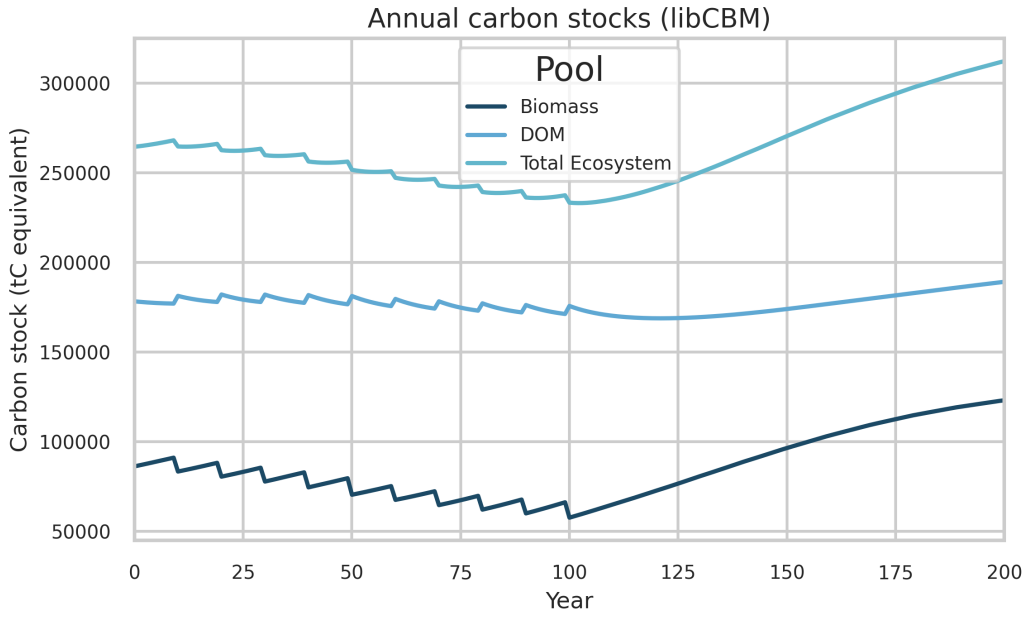


Figure 7: libCBM results: annual carbon stocks (biomass, dead organic matter, total ecosystem) over 200 years.

preserved in the Zenodo-archived release.

Broader example library and carbon-aware optimization

Beyond the sequential ~~carbon-accounting~~ carbon accounting demonstration, WS3 includes ~~a curated set of executable notebooks designed as progressive tutorials , from synthetic examples to realistic~~ executable notebooks that serve as tutorials and validation exercises, ranging from synthetic cases to policy-oriented analyses. Example 040 implements the ~~classic Neilson hack from scratch and validates that~~ Neilson hack and compares aggregated WS3 carbon indicators ~~closely track~~ with libCBM in a no-disturbance scenario. ~~Carbon-aware optimization (~~Example 041 illustrates carbon-aware optimization by embedding carbon prices/constraints ~~directly~~ in the LP and evaluating schedules with `libcbm_py`~~) is illustrated in Example 041.~~ .

4. Impact and reusability

WS3 ~~enables transparent, auditable analysis for forest landscape planning and climate mitigation, responding to calls for credible natural climate solutions and land-sector mitigation accounting~~

[18, 15]. It has supported has been used in research on bioenergy [7], value-creation potential and supply modelling modeling [40, 39], and climate impact assessment [46, 27, 53]. WS3 It integrates with the SpaDES ecosystem for spatial simulation [8] and has been deployed as a backend in decision-support applications. The open MIT license , modular API , and PyPI distribution lower barriers to community contributions and MIT license and modular API support reuse across jurisdictions.

Major use-cases use cases (2015–2025).

- Strategic wood supply and bioenergy planning in mixed-wood forests: optimization and scenario analysis of value-creation options [7].
- Hybrid simulation–optimization for value indicators and model retrofitting: methods and transfer to production-style analyses [40, 39].
- Climate mitigation and carbon-aware planning in British Columbia: sequential WS3→libCBM pipelines and optimization under policy constraints; graduate theses and applied demonstrations [27, 53, 52].
- Decision-support prototypes for nature-based solutions: avoided fire and avoided harvest workflows (Examples 050/060) with net-emission indicators derived from libCBM.
- Integration with spatial simulation: `spades_ws3` module bridging WS3 with SpaDES for disturbance and landscape dynamics studies [8, 50].
- Application backends: WS3 used in web-based decision-support systems (e.g., `ecotrust-dss`) for scenario configuration and reporting [49].
- Ongoing initiatives: CCCANDiES (integrated carbon and ecosystem services; in progress) and mining-sector nature-based solutions [16].

Interoperability fosters reusability: is supported through tabular/text inputs, raster outputs, and the libCBM linkageallow , enabling WS3 to slot into integrate with existing analytical pipelines. The reproduction package , unit tests , and continuous integration (CI) pipelines enhance trust and and unit tests facilitate method transfer , aligned with community guidance on open and efficient scientific practice and support open scientific practices [23, 31]. The architecture is intentionally

~~general , making it applicable~~ general and has been applied to studies of sustainable yield, carbon accounting, biodiversity, and policy trade-offs [2].

~~*Practical impact and reusability highlights*~~*Reusability features*.

- Openness and packaging: MIT license~~, PyPI distribution, ~~; versioned releases archived on Zenodo [38]; complete examples and tutorials.
- Interoperability: Woodstock-format import for legacy/industry pipelines; libCBM SIT export for carbon; standard geospatial formats (GeoTIFF, ~~shapefile~~shapefiles).
- Reproducibility: deterministic reproduction package (~~`repro`~~`repro`) that regenerates all figures ~~/tables; continuous integration (CI) runs tests and validates~~ and tables; tests validate examples.
- Extensibility: modular API for adding outputs/constraints and new scheduling logic (heuristic or LP-based); ~~solver-agnostic~~ solver-flexible (HiGHS by default; Gurobi optional).
- Integration pathways: SpaDES ecosystem via `spades_ws3`; backend for decision-support applications and web services.
- Education and onboarding: progressive example notebooks (from `010` to `060`) facilitate training and method transfer.

~~An advanced example notebook demonstrates how WS3 can be used to construct~~ Example 040 demonstrates a stand-level optimization ~~problem~~ workflow with explicit integration of carbon dynamics using `libcbm_py`. ~~This example solves a linear program that balances timber harvest revenues with carbon sequestration benefits under different carbon pricing assumptions (see Example 040). The workflow illustrates~~ The workflow shows how WS3's optimization layer and libCBM coupling ~~enable~~ can support carbon-aware planning experiments ~~without bespoke glue code, supporting emerging policy analyses in nature-based climate solutions~~while leaving the precise objective formulation to the user.

5. Comparison to alternatives

Parity with Woodstock. To ~~reassure readers~~ demonstrate that WS3 implements standard inventory accounting correctly, we report a parity check against a ~~prescriptive~~ reference Woodstock

Table 3: Comparison of WS3 with selected tools.

Tool	Scope	License	Optimization	Spatial	Carbon	Region
WS3	Landscape	Open (MIT)	Built-in (PuLP; optional Gurobi)	Hybrid aspatial-to-raster	Built-in libCBM linkage	General
Woodstock	Landscape	Proprietary	Built-in	Extensions; connectors	External integrations	General
Patchworks	Landscape	Proprietary	Built-in	Spatial planning emphasis	External integrations	General
SpaDES	Simulation framework	Open	External packages	Fully spatial (agent-based, raster)	Via modules	General
~~SIMFOR~~	Landscape	Restricted (mixed)	Built-in	Spatial capabilities	External integrations	EU-focused
~~Stand-level~~	~~Open~~	~~Limited/none~~	~~N/A~~	~~Limited/none~~	~~General~~	Heureka
JLP (MELA)	Landscape	Restricted (legacy)	Built-in	Spatial extensions	External integrations	Finland-focused

run using the same toy dataset and an identical action schedule. This test is a transparent, reproducible illustration rather than an exhaustive proof of equivalence. The accompanying CSV artifact `woodstock_parity.csv` provides total harvest area, harvest volume, and growing stock, along with WS3-relative percent differences (computed as $100\,(\text{WS3}-\text{Woodstock})/\text{Woodstock}$). For transparency, period-wise parity summaries for harvest area and volume can be regenerated via the reproduction scripts and are included in the reproduction package ~~.~~(Figure 9). Exact or near-exact agreement is expected given identical inputs.

WS3 ~~differs primarily in its~~ provides an open Python API, ~~explicit~~ a documented libCBM integration, and a hybrid spatial/aspatial workflow ~~designed for reproducible research and policy analysis. We emphasize that SIMFOR operates at stand level, while WS3,~~ intended for reproducible analyses. It is a framework rather than a GUI decision-support suite; spatial adjacency, opening-size constraints, and road-network planning are outside its scope. Woodstock, Patchworks, Heureka, and JLP target landscape-scale planning with varying degrees of spatial explicitness and availability

Table 4: WS3 vs. Woodstock parity on the toy dataset (identical schedule). Values are loaded from the accompanying CSV artifact.

Metric	WS3	Woodstock	WS3 vs Woodstock (%)
Total harvest area (ha)	1,000.00	1,000.00	0.00
Total harvest volume (m^3)	164,838.08	164,838.08	0.00
Total growing stock (m^3)	1,269,692.15	1,269,692.15	0.00

~~[44, 47, 8, 24, 35, 45]~~[44, 47, 8, 24, 35].

Comparison narrative. Proprietary landscape systems such as Woodstock and Patchworks ~~provide~~ offer mature optimization and spatial planning capabilities, including native support for adjacency and block constraints, but their licensing and closed codebases can limit transparent methods development ~~, automated testing,~~ and reproducibility at scale. SpaDES ~~, by contrast,~~ is an open simulation framework oriented to spatial processes and agent-based dynamics; it does not natively provide a solver-backed harvest scheduling layer but ~~interoperates~~ can interoperate with WS3 via `spades_ws3` for hybrid workflows. Heureka and JLP are landscape-oriented and include optimization, yet are region-specific or legacy-maintained, which constrains adoption beyond their primary jurisdictions.

Carbon-aware planning and optimization have an active literature that couples timber supply models with carbon accounting and evaluates mitigation trade-offs under alternative schedules and policies. Early integrations such as CO2Fix paired with timber supply modeling illustrate carbon-informed planning workflows [36]. More recent sector and landscape studies quantify mitigation potential and carbon capacities under harvest scenarios [46, 2], and graduate theses in British Columbia explicitly evaluate climate objectives in forest planning optimization contexts [27, 53]. WS3 ~~'s distinctive contribution is to combine~~provides a reproducible, open framework for constructing these workflows while keeping the scheduling engine transparent and auditable.

WS3 integrates: (i) a ~~solver-agnostic~~solver-flexible, path-based Model I LP generator embedded ~~directly~~ in an open forest estate API; (ii) a documented ~~, first-class~~ linkage to libCBM for carbon stocks and fluxes; and (iii) a hybrid spatial/aspatial pipeline with standard geospatial I/O. ~~These design choices enable reproducible , auditable~~ This design supports reproducible workflows in notebooks and scripts ~~, fast iteration on objectives/constraints, and easy integration into~~ and integrates

with larger research and decision-support ecosystems. ~~For many research and policy analyses, this balance of openness, extensibility, and end-to-end reproducibility is decisive—even when spatial planning itself is conducted in a specialized tool,~~ Where spatially explicit optimization is required, WS3 ~~can serve as the transparentscheduling and carbon accountingengine feeding it~~can provide schedules and carbon accounting inputs to specialized spatial tools; carbon-related objectives or constraints can be added when users supply coefficients, but they are not required for the core workflow demonstrated here.

6. Discussion

WS3 is intended as an open, reproducible alternative to proprietary forest estate planning systems. Its contribution is primarily informatics: a transparent, scriptable workflow that integrates scheduling, carbon accounting, and spatial reporting, rather than a new optimization formulation.

7. ~~Limitations and scope~~

6.1. Scope and limitations

WS3 focuses on deterministic, aspatial forest estate formulations. It does not attempt to represent stochastic risk, uncertainty propagation, machine learning forecasting, or fully spatial optimization; those are handled through scenario ensembles or specialized tools rather than within the core framework. The Model I LP implementation is well suited to even-flow policies, policy bounds, and ~~value-maximisation~~ value maximization problems, yet it inherits ~~the~~ classic limitations of the formulation: stochastic or ~~endogenous~~ exogenous disturbances cannot be represented faithfully without ~~departing from linearity~~breaking the required certainty assumption of linear programming (this is not an issue when running purely prescriptive sequential simulations or scheduling actions using priority-queue heuristic mode), and non-linear objectives or constraints require either ~~linearisation~~ linearization or external wrappers. In practice, we ~~rely on~~ use the heuristic scheduler to ~~interleave random disturbances~~ run disturbance realizations or sensitivity analyses when scenario logic demands it, and reserve the LP for static policy experiments.

Spatial representation is ~~delivered through a post-hoc~~ provided through a post hoc raster allocation step that respects operability and transitions but does not enforce spatial adjacency, road-building, or block-shape constraints. Those dynamics fall outside the scope of WS3~~and are the~~

~~motivation for ongoing spatial extensions (e.g., the forthcoming WS4 prototype)~~. Users requiring spatial ~~optimisation should therefore~~ optimization should either select a different modelling framework, or treat WS3 as a scheduling and accounting engine ~~that feeds specialised spatial tools~~(that can sit either upstream or downstream from more specialized spatial modelling tools, depending on workflow and project objectives).

In this manuscript, carbon accounting is applied post hoc to schedules produced by the LP or heuristic workflows. The framework can be extended to incorporate carbon-related objectives or constraints when users supply the necessary coefficients, but those formulations are not the focus of the present study.

Finally, large LP instances involve substantial memory pressure—tens of gigabytes for the largest benchmarks—because the column generation compiles full path matrices. The reproduction package flags the LP scaling experiment as opt-in so that teams can execute it on appropriately provisioned infrastructure. All supporting assets ~~, however,~~ remain deterministic and auditable even when the LP portion is skipped.

7. Conclusions and future work

WS3 provides an open, ~~extensible~~ reproducible framework for integrating harvest scheduling, simulation, ~~and carbon accounting~~carbon accounting, and spatial reporting. By coupling solver-backed scheduling with libCBM via a documented interface, WS3 supports ~~policy-relevant analyses with transparent , reproducible workflows~~transparent Model I analyses that can be inspected, reproduced, and extended.

Limitations ~~of the current release centre on input data fidelity and computational resources. WS3 assumes that yield curves~~remain: results depend on the quality of user-supplied yields, transition rules, and carbon parameters~~supplied by users are defensible; errors in these inputs propagate directly into optimization outcomes and libCBM flux estimates. The deterministic Model I formulation also omits stochastic disturbance processes~~; stochastic disturbances and climate-forced growth uncertainty ~~, so scenario ensemblesare still required to bracket plausible futures. Finally, large linear programs can demand tens of gigabytes of memoryand access to multi-core CPUs; while the reproduction package identifies these hotspots, practitioners should budget HPC or cloud resources accordingly~~require external scenario ensembles; and large LP

instances can require substantial memory. The reproduction package highlights these constraints and provides deterministic assets for audit and reuse.

Future work will ~~expand uncertainty handling (stochastic scheduling, sensitivity analysis), add cloud/HPC deployment patterns, and strengthen spatial allocation and visualization. We also plan to enhance carbon–economics linkages and support machine learning surrogates for speedups in large scenario ensembles [12]. The open design invites community extensions and cross-ecosystem applications~~prioritize uncertainty handling and spatial extensions, guided by community contributions to the open codebase.

Reproducibility. All figures and tables are generated by scripts in ~~repro~~`repro`. The `make_repro.sh` script ~~creates~~ provisions an isolated environment, installs pinned dependencies (`requirements.txt`), and runs `generate_diagrams.py`, the spatial allocation workflow (`generate_spatial_allocation.py`), and the sequential demonstration script (`generate_case_study.py`). Outputs are written to ~~figs and tables~~`figs` and `tables`. The exact versions are preserved via the Zenodo-archived release.

FAIR compliance. Table 5 ~~summarises the EMS~~ summarizes the FAIR/software checklist submitted with the manuscript (see supplementary CSV `fair_checklist.csv`). WS3 remains Findable, Accessible, Interoperable, and Reusable through its Zenodo DOI archive and indexed repositories, ~~Accessible under the MIT license with public~~ MIT-licensed code/data ~~artefacts, Interoperable via~~ artifacts, open tabular and geospatial formats plus Woodstock/libCBM linkages, and ~~Reusable through pinned environments, continuous-integration (CI)–tested workflows, and~~ full reproduction scripts [38].

CRediT authorship contribution statement

G.E.P.: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing—original draft, Writing—review & editing, Visualization, Supervision, Project administration, Funding acquisition.

Acknowledgements

Development of WS3 was supported by the Forest Resources Environmental Services Hub (FRESH) at the University of British Columbia. The author thanks Elaheh Ghasemi, Salar Ghotb,

Table 5: ~~EMS~~ FAIR/software checklist summary for WS3.

Principle	Checklist focus	Evidence / ~~artefact~~ artifact
Findable	Persistent identifier; indexed repository	Zenodo DOI 10.5281/zenodo.17331213; tagged GitHub releases; ~~PyPI project~~ indexed repository metadata
Accessible	License and public access	MIT license (CITATION.cff, LICENSE); public GitHub repository; ~~PyPI wheels;~~ reproduction package assets bundled in repository
Interoperable	Open formats and connectors	Woodstock LAN/ARE/YLD importers; libCBM SIT export; tabular CSV, GeoTIFF, and shapefile outputs; documented workflow in Figure 1
Reusable	Documentation, provenance, validation	ReadTheDocs site; notebooks/examples; ~~continuous-integration (CI) tested~~ `pytest` suite; deterministic scripts in ~~repro~~`repro`; submission checklist metadata

and Kathleen Coupland for ~~their~~ contributions to documentation, testing, and code quality improvements~~leading up to this release~~. Language polishing and compliance checks were assisted by the OpenAI GPT-5 and GPT-5-Codex models; the author reviewed and edited all outputs.

Funding

~~Funding:~~ Natural Sciences and Engineering Research Council of Canada (NSERC, grant RGPIN-2023-04197); Environment and Climate Change Canada (ECCC, grant 3000770190); Natural Resources Canada (NRCan, grant 3000771054); Government of British Columbia (grant TP24FCCS004); Mitacs (Newmont Goldcorp Inc., grant IT32088).

Data availability

~~Data availability:~~ Source code, configuration files, and reproduction assets are archived at ~~https://doi.org/10.5281/zenodo.17331213~~Zenodo (DOI: `10.5281/zenodo.17331213`). All scripts used to generate the figures and tables are included; reuse is permitted under the MIT License.

Declaration of competing interest

~~Competing interests:~~ The author declares no competing interests.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used the OpenAI GPT-5 and GPT-5-Codex models to assist with language polishing and format compliance. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

[1] Apex Resource Management Solutions, 2024. SyncroSim: Scenario Modeling Platform [software]. URL: https://syncrosim.com. accessed 2025-09-28.

[2] Boisvenue, C., Paradis, G., Eddy, I.M., McIntire, E.J., Chubaty, A.M., 2022. Managing forest carbon and landscape capacities. Environmental Research Letters 17, 114013. doi:10.1088/1748-9326/ac9919.

[3] Canadell, J.G., Monteiro, P.M.S., Joos, F., et al., 2022. Agriculture, forestry and other land use, in: Shukla, P.R., Skea, J., Slade, R., et al. (Eds.), Climate Change 2022: Mitigation of Climate Change. Contribution of Working Group III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change. Cambridge University Press, Cambridge, UK, pp. 751–850. URL: https://keneamazon.net/Documents/Publications/Virtual-Library/Impacto/157.pdf.

[4] Canadian Forest Service, 2025a. libcbm-py: Python implementation of the Carbon Budget Model [software]. URL: https://pypi.org/project/libcbm/. Python package (accessed 2025-09-28).

[5] Canadian Forest Service, 2025b. libcbm_py: Carbon Budget Model Python Interface [software]. URL: https://github.com/cat-cfs/libcbm_py. source code repository (accessed 2025-09-28).

[6] Canadian Forest Service, 2025c. libcbm_py Documentation. URL: https://cat-cfs.github.io/libcbm_py/. documentation site. (accessed 2025-09-28).

[7] Cantegril, P., Paradis, G., LeBel, L., Raulier, F., 2019. Bioenergy production to improve value-creation potential of strategic forest management plans in mixed-wood forests of eastern canada. Applied Energy 247, 171–181. doi:10.1016/j.apenergy.2019.04.022.

[8] Chubaty, A.M., McIntire, E.J.B., et al., 2024. SpaDES: Spatially Explicit Discrete Event Simulation [software]. URL: https://spades.predictiveecology.org. r package and framework (accessed 2025-09-28).

[9] Daigneault, A.J., Hayes, D.J., Fernandez, I.J., Weiskittel, A.R., 2022. Forest carbon accounting and modeling framework alternatives: an inventory, assessment, and application guide for Eastern US State Policy Agencies. Technical Report. University of Maine, Center for Research on Sustainable Forests. URL: https://crsf.umaine.edu/resource/forest-carbon-accounting-and-modeling-guide/.

[10] Feunekes, U., Cogswell, A., 1997. A hierarchical approach to spatial forest planning. General Technical Report NC-205. USDA Forest Service, North Central Forest Experiment Station. St. Paul, MN, USA. URL: https://remsoft.com/wp-content/uploads/2020/06/Remsoft_SSAFR-1997_-Hierarchical-Approach-to-Spatial-Forest-Planning.pdf.

[11] Food and Agriculture Organization of the United Nations, 2020. Global Forest Resources Assessment 2020. Main Report. Technical Report. Food and Agriculture Organization of the United Nations. Rome. doi:10.4060/ca9825en.

[12] Forrester, A.I.J., Sobester, A., Keane, A.J., 2008. Engineering Design via Surrogate Modelling: A Practical Guide. John Wiley & Sons. doi:10.1002/9780470770801.

[13] Fortin, M., Lavoie, J.F., 2022. Reconciling individual-based forest growth models with landscape-level studies through a meta-modelling approach. Canadian Journal of Forest Research 52, 1140–1153. doi:10.1139/cjfr-2022-0002.

[14] Fortin, M., Sattler, D., Schneider, R., 2022. An alternative simulation framework to evaluate the sustainability of annual harvest on large forest estates. Canadian Journal of Forest Research 52, 704–715. doi:10.1139/cjfr-2021-0255.

[15] Gasser, T., Crepin, L., Quilcaille, Y., Houghton, R.A., Ciais, P., Obersteiner, M., 2020. Historical co2 emissions from land use and land cover change and their uncertainty. Biogeosciences 17, 4075–4101. doi:10.5194/bg-17-4075-2020.

[16] Ghasemi, E., Coupland, K., Ghotb, S., Paradis, G., 2025. Developing a prototype decision support framework to assess forest management scenarios as a nature-based decarbonization solution for the mining sector: A case study in british columbia, canada. EarthArXiv. URL: https://eartharxiv.org/repository/view/10522/, doi:10.31223/X5W731.

[17] Gilson, L.W., Eskelson, B.N.I., Sattler, D.F., 2025. What makes a forest growth model climate-sensitive? an examination of statistical and silvicultural model needs under climate change. Forestry 98, 649–660. doi:10.1093/forestry/cpaf020.

[18] Griscom, B.W., Adams, J., Ellis, P.W., et al., 2017. Natural climate solutions. Proceedings of the National Academy of Sciences 114, 11645–11650. doi:10.1073/pnas.1710465114.

[19] Gunn, E., 2007. Models for Strategic Forest Management, in: Weintraub, A., Romero, C., Bjørndal, T., Epstein, R. (Eds.), Handbook of Operations Research in Natural Resources. Springer Science+Business Media. chapter 16, pp. 317–341. doi:10.1007/978-0-387-71815-6_16.

[20] Gunn, E., 2009. Some perspectives on strategic forest management models and the forest products supply chain. INFOR: Information Systems and Operational Research 47, 261–272. doi:10.3138/infor.47.3.261.

[21] Gurobi Optimization, LLC, 2024. Gurobi Optimizer [software]. URL: https://www.gurobi.com. accessed 2025-09-28.

[22] Hall, J., et al., 2023. HiGHS: High performance software for linear optimization [software]. URL: https://highs.dev/. accessed 2025-09-28.

[23] Hampton, S.E., Anderson, S.S., Bagby, S.C., et al., 2015. The tao of open science for ecology. Ecosphere 6, 1–13. doi:10.1890/ES14-00402.1.

[24] Heureka Consortium, 2024. Heureka Forestry Decision Support System [software]. URL: https://www.heurekaslu.se/. accessed 2025-09-28.

[25] Jamnick, M.S., Walters, K.R., 1993. Spatial and temporal allocation of stratum-based harvest schedules. Canadian Journal of Forest Research 23, 402–413. doi:10.1139/x93-058.

[26] Johnson, K.N., Scheurman, H.L., 1977. Techniques for prescribing optimal timber harvest and investment under different objectives: discussion and synthesis. Forest Science 23. doi:10.1093/forestscience/23.s1.a0001. monograph 18.

[27] Ke, J., 2024. Climate impact assessment under different forest harvesting and fertilization scenarios. Master's thesis. University of British Columbia. doi:10.14288/1.0441338.

[28] Kurz, W., Dymond, C., Stinson, G., Rampley, G., et al., 2009. CBM-CFS3: A model of carbon-dynamics in forestry and land use change. Canadian Journal of Forest Research 39, 495–509. doi:10.1016/j.ecolmodel.2008.10.018.

[29] LANDIS-II Foundation, 2024. LANDIS-II Forest Landscape Model [software]. URL: https://www.landis-ii.org. accessed 2025-09-28.

[30] Law, B.E., Hudiburg, T.W., Berner, L.T., 2018. Land use strategies to mitigate climate change in carbon dense temperate forests. Proceedings of the National Academy of Sciences 115, 3663–3668. doi:10.1073/pnas.1720064115.

[31] Lowndes, J.S.S., Best, B.D., Scarborough, C., et al., 2017. Our path to better science in less time using open data science tools. Nature Ecology & Evolution 1, 0160. doi:10.1038/s41559-017-0160.

[32] McIntire, E.J., Chubaty, A.M., Cumming, S.G., Andison, D., Barros, C., Boisvenue, C., Haché, S., Luo, Y., Micheletti, T., Stewart, F.E., 2022. PERFICT: A Re-imagined foundation for predictive ecology. Ecology Letters 25, 1345–1351. doi:10.1111/ele.13994.

[33] Mistik Management Ltd., 2019. 2019 20-year forest management plan: Volume II Document V – Modeling assumptions. Technical Report. Mistik Management Ltd.. Meadow Lake, Saskatchewan, Canada. URL: https://www.mistik.ca/wp-content/uploads/2019-Documents/Vol_II_Doc5_Modeling_Assumptions.pdf.

[34] Mitchell, S., et al., 2023. PuLP: A Linear Programming Toolkit for Python [software]. URL: https://coin-or.github.io/pulp/. accessed 2025-09-28.

[35] Natural Resources Institute Finland (Luke), 2024. JLP (MELA) Forest Planning System. URL: http://mela2.metla.fi/mela/j/jlp-en.htm. accessed 2025-09-28.

[36] Neilson, E.T., MacLean, D.A., Arp, P.A., Meng, F.R., Bourque, C.P., Bhatti, J.S., 2006. Modeling carbon sequestration with CO2Fix and a timber supply model for use in forest management planning. Canadian Journal of Soil Science 86, 219–233. doi:10.4141/S05-081.

[37] Nguyen, D., Henderson, E., Wei, Y., 2022. Prism: A decision support system for forest planning. Environmental Modelling & Software 157, 105515. doi:10.1016/j.envsoft.2022.105515.

[38] Paradis, G., 2025. ws3: Wood Supply Simulation System [software]. doi:10.5281/zenodo.17331213.

[39] Paradis, G., Lebel, L., 2018a. Estimating the value-creation potential of wood supply using a hybrid simulation-optimization approach. Technical Report CIRRELT-2018-24. Centre interuniversitaire de recherche sur les réseaux d'entreprise, la logistique, et le transport (CIRRELT). doi:10.13140/RG.2.2.32706.48321.

[40] Paradis, G., Lebel, L., 2018b. Retro-Fitting Value-Creation Potential Indicators to Long-Term Supply Models. Technical Report CIRRELT-2018-23. Centre interuniversitaire de recherche sur les réseaux d'entreprise, la logistique, et le transport (CIRRELT). doi:10.13140/RG.2.2.19284.71040.

[41] Paradis, G., LeBel, L., D'Amours, S., Bouchard, M., 2013. On the risk of systematic drift under incoherent hierarchical forest management planning. Canadian Journal of Forest Research 43, 480–492. doi:10.1139/cjfr-2012-0334.

[42] Peng, R.D., 2011. Reproducible research in computational science. Science 334, 1226–1227. doi:10.1126/science.1213847.

[43] Power, H., 2015. Guide d'utilisation du simulateur de croissance forestière Artémis-2014 sur Capsis (Version préliminaire 3.1). Technical Report. Ministère des Forêts, de la Faune et des Parcs, Gouvernement du Québec. Québec, Canada. URL: https://mrnf.gouv.qc.ca/documents/forets/connaissances/recherche/Guide-Artemis-Capsis.pdf. direction de la recherche forestière.

[44] Remsoft Inc., 2025. Woodstock [software]. URL: https://www.remsoft.com/products/woodstock/. accessed 2025-09-28.

[45] SIMFOR Project, . SIMFOR: A stand-level simulation tool. Latexdiff stub entry for removed citation.

[46] Smyth, C., Xu, Z., Lemprière, T., Kurz, W., 2020. Climate change mitigation in British Columbia's forest sector: GHG reductions, costs, and environmental impacts. Carbon Balance and Management 15, 1–22. doi:10.1186/s13021-020-00155-2.

[47] Spatial Planning Systems Inc., 2025. Patchworks [software]. URL: https://www.spatial.ca/patchworks/. accessed 2025-09-28.

[48] UBC FRESH Lab, 2024. Ws3 continuous integration workflow. https://github.com/UBC-FRESH/ws3/blob/main/.github/workflows/ci.yml. URL: https://github.com/UBC-FRESH/ws3/blob/main/.github/workflows/ci.yml. accessed 2025-09-28.

[49] UBC-FRESH Lab, 2025a. ecotrust-dss: WS3-backed decision support application. doi:10.5281/zenodo.17330007.

[50] UBC-FRESH Lab, 2025b. spades_ws3: Linking WS3 harvest scheduling with SpaDES [software]. doi:10.5281/zenodo.17330027.

[51] University of British Columbia, n.d. Forest Planning Studio ATLAS (FPS-ATLAS) [software]. Historical software; development discontinued.

[52] Yan, Y., 2024. msc_walter_yan: Carbon-aware harvest scheduling case study [software]. URL: https://github.com/UBC-FRESH/msc_walter_yan. gitHub repository (accessed 2025-09-28).

[53] Yan, Y., 2025. A Mathematical Modeling Framework to Optimize the Climate Change Mitigation Potential of the Forest Sector in British Columbia, Canada. Master's thesis. University of British Columbia. doi:10.14288/1.0449036.

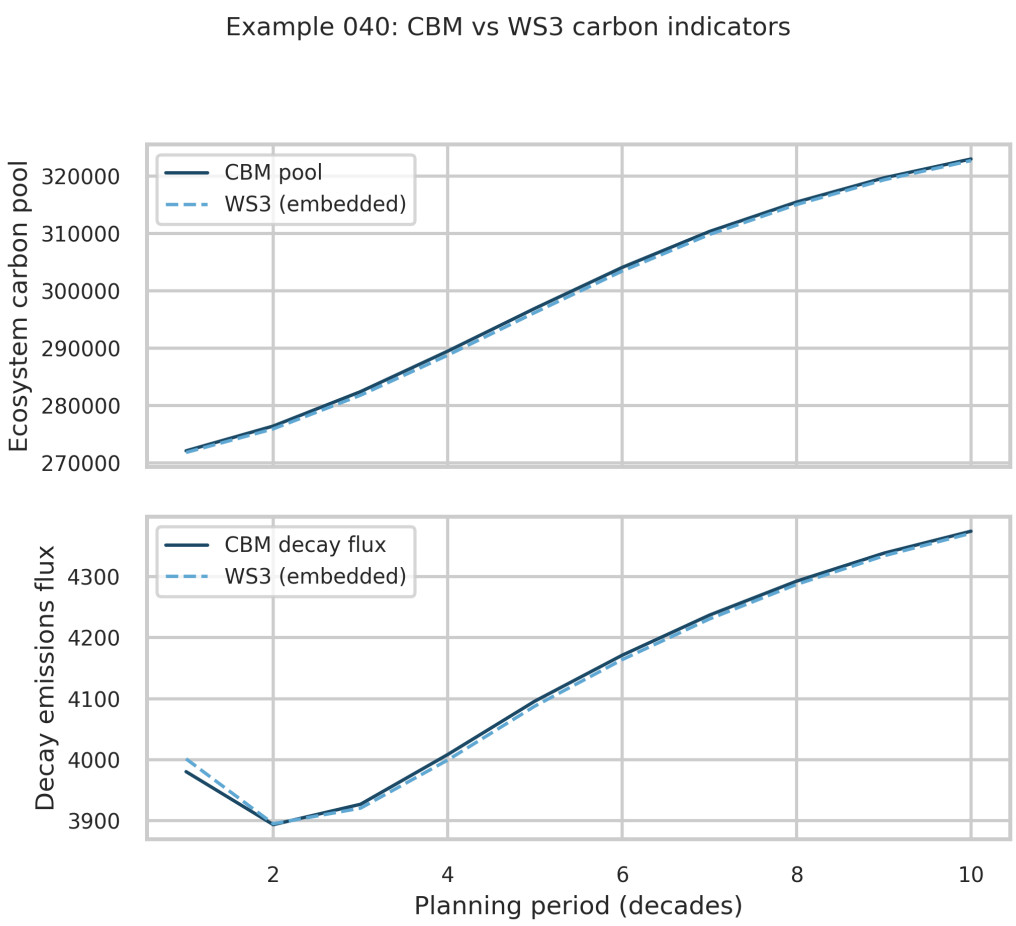


Figure 8: Neilson-hack demonstration (Example 040): comparison of libCBM (solid) and WS3-embedded (dashed) carbon indicators after bootstrapping pools/fluxes into WS3 as yield curves. Top: ecosystem carbon pools; ~~Bottom~~bottom: aggregate decay emissions flux. In a no-disturbance scenario, the aggregated indicators match closely; with harvesting, flux gaps reflect known limitations of the Neilson hack due to CBM DOM spin-up assumptions mapping time to age (Figure 8).

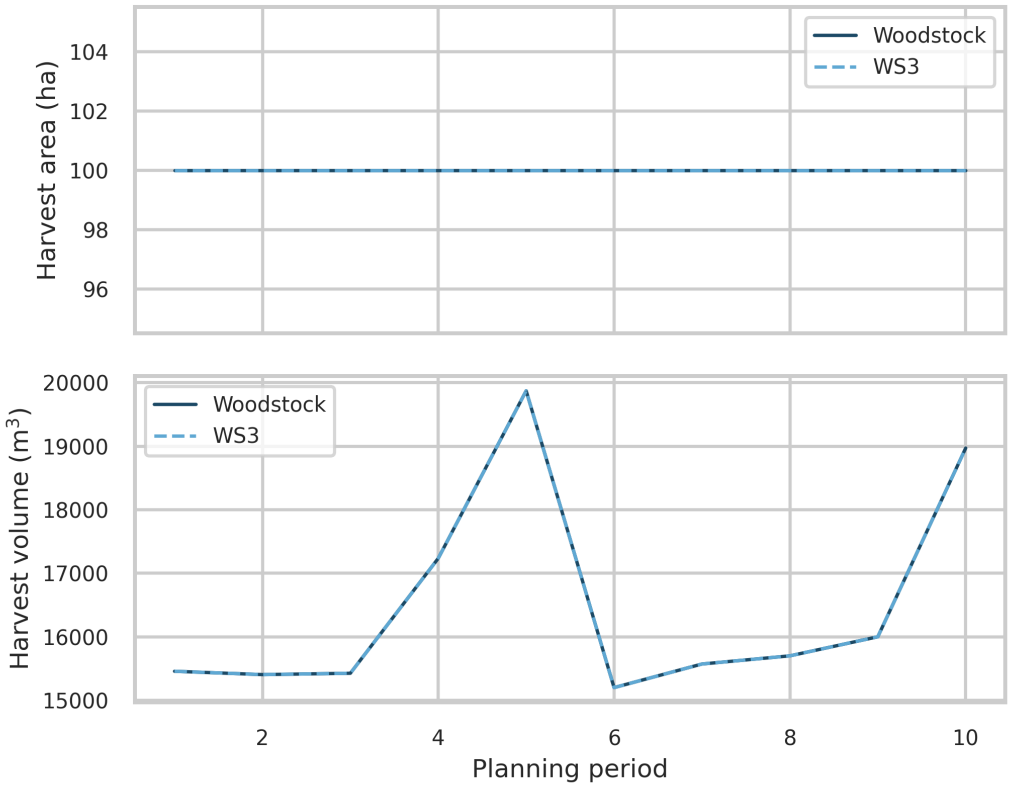


Figure 9: Period-wise parity of harvested area and harvested volume between WS3 and Woodstock on the toy dataset (identical schedule). Values are loaded from the accompanying CSV artifact.